

CTG-CPM: MASTER TECHNICAL MANUAL

Self-Healing Networks & Host Predictive Maintenance via Counterfactual Telemetry

Project: CTG-CPM Self-Healing System	Version: 2.0 (LLM Integrated)
Authors: Sagnik Basu, C Sriharsha, Maitree Singh	Target MTTR: < 10 Seconds

CTG-CPM: Master Technical Guide & Viva Preparation Manual

Self-Healing Networks & Host Predictive Maintenance via Counterfactual Telemetry

Table of Contents

- [Executive Summary & Core Concept](#1-executive-summary--core-concept)
- [Data Layer: What Data is Used & How It is Collected](#2-data-layer-what-data-is-used--how-it-is-collected)
- [The 5-Layer System Architecture](#3-the-5-layer-system-architecture)
- [Deep-Dive into Algorithms & Internal Mechanics](#4-deep-dive-into-algorithms--internal-mechanics)
- [Algorithm 1: GraphSAGE Graph Neural Network (GNN)](#algorithm-1-graphsage-graph-neural-network-gnn)
- [Algorithm 2: Time-Series Diffusion Model (Diffusion-TS)](#algorithm-2-time-series-diffusion-model-diffusion-ts)
- [Algorithm 3: Shapley Value Root-Cause Attribution](#algorithm-3-shapley-value-root-cause-attribution)
- [Algorithm 4: VCG Auction for Agent Task Allocation](#algorithm-4-vcg-auction-for-agent-task-allocation)
- [Algorithm 5: Extensive-Form Game & Backward Induction (SPE)](#algorithm-5-extensive-form-game--backward-induction-spe)
- [Algorithm 6: Nash Equilibrium Conflict Resolution](#algorithm-6-nash-equilibrium-conflict-resolution)
- [Algorithm 7: LLM-Powered Diagnostics (Groq Engine)](#algorithm-7-llm-powered-diagnostics-groq-engine)
- [End-to-End Execution Flow (Step-by-Step Scenario)](#5-end-to-end-execution-flow-step-by-step-scenario)
- [Codebase Map & Module Reference](#6-codebase-map--module-reference)
- [Viva & Review Q&A Cheatsheet (Top 15 Questions & Answers)](#7-viva--review-qa-cheatsheet-top-15-questions--answers)
- [Individual Contributions Breakdown](#8-individual-contributions-breakdown)

- **Data points collected:**
- `cpu_overall_percent`: Overall CPU utilization across all cores.
- `cpu_per_core`: Array of individual CPU core loads.
- `cpu_frequency_mhz`: Operating CPU clock speed.
- `memory_percent` & `memory_used_mb`: RAM consumption.
- `disk_read_kbps` & `disk_write_kbps`: I/O throughput.
- `top_processes`: List of top CPU-consuming process names, PIDs, and usage %.

2. 5G Optical Backhaul Telemetry (Simulated)

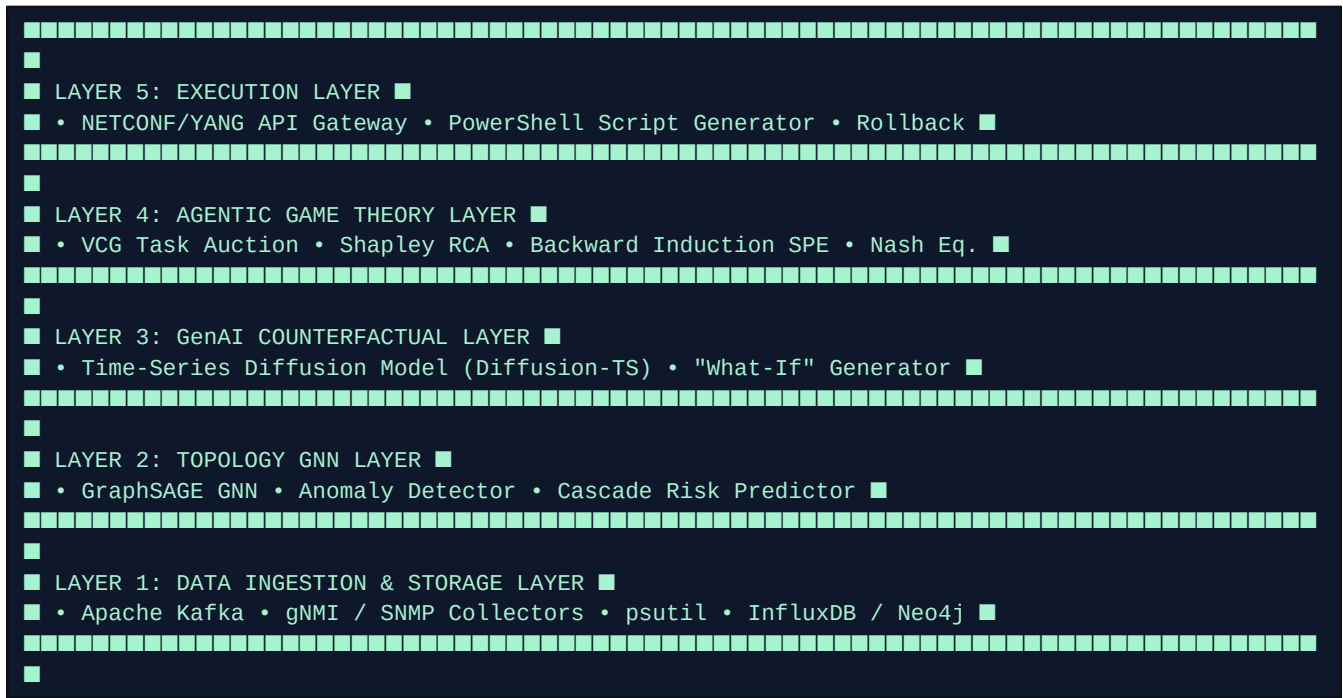
- **Data points generated:**
- `osnr_db`: Optical Signal-to-Noise Ratio (Normal: ~22.4 dB, Anomaly: < 18.0 dB).
- `laser_bias_ma`: Current powering the optical laser (Normal: ~45 mA, Anomaly: > 65 mA).
- `temperature_celsius`: Physical transceiver temp (Normal: ~52°C, Anomaly: > 75°C).
- `packet_loss_percent`: Percentage of lost data packets (Normal: 0.01%, Anomaly: > 3.0%).

3. Industrial Data Architecture (Production Setup)

In a full enterprise telecommunication or cloud data center deployment:

- **Streaming Telemetry Bus**: Apache Kafka / gNMI (gRPC Network Management Interface).
- **Time-Series Database**: InfluxDB or Prometheus for metrics storage.
- **Graph Database**: Neo4j for network topology graph storage.

3. The 5-Layer System Architecture



Layer-by-Layer Breakdown:

- **Layer 1 (Data Layer):** Continuously streams raw telemetry metrics from physical servers or optical transceivers into real-time storage.
- **Layer 2 (Topology Layer):** Uses a Graph Neural Network (GraphSAGE) to understand how devices are connected physically and detect when a metric strays outside safe thresholds.
- **Layer 3 (GenAI Layer):** When an anomaly occurs, Generative AI creates parallel "What-If" futures representing different possible fix actions.
- **Layer 4 (Agentic Game Theory Layer):** Multi-Agent AI coordinates using formal mathematical rules (VCG, Shapley, SPE, Nash) to select the optimal fix without agent conflicts.
- **Layer 5 (Execution Layer):** Converts the verified fix into executable commands (NETCONF XML for routers, PowerShell for Windows) and applies it safely.

4. Deep-Dive into Algorithms & Internal Mechanics

Algorithm 1: GraphSAGE Graph Neural Network (GNN)

- **Role:** Topology-Aware Anomaly Detection & Cascade Risk Prediction.
- **Why this over standard CNNs/MLPs?**

- Computer networks and data center servers are **graphs** (nodes connected by links), not 2D grids (images). Standard CNNs assume grid structures and fail on graph data.
- GraphSAGE samples neighboring node features and aggregates them into node embeddings.

- **How it works internally:**

- Each device in the network is a node v .
- GraphSAGE gathers telemetry features from node v 's connected neighbors:

$$h_{N(v)}^k = \text{AGGREGATE}_k(\{h_u^{k-1}, \forall u \in N(v)\})$$

- It combines neighbor information with node v 's own state to form an updated representation h_v^k .
- If node v 's embedding strays beyond normal bounds, an anomaly is flagged, and GraphSAGE predicts which downstream connected nodes are at risk of cascading failure.

Algorithm 2: Time-Series Diffusion Model (Diffusion-TS)

- **Role:** Counterfactual Telemetry Generation ("What-If" Future Simulator).
- **Why this over GANs (Generative Adversarial Networks)?**
- GANs suffer from **mode collapse** (they get stuck generating only one type of output) and are notoriously unstable to train on continuous time-series metrics.
- Diffusion models work via **denoising score matching**: they learn to reverse a gradual noise process, producing stable, high-fidelity synthetic time-series data.
- **How it works internally:**
- **Forward Process (Adding Noise):** Takes baseline telemetry x_0 and gradually adds Gaussian noise over T steps until it becomes pure noise x_T .
- **Reverse Process (Denoising):** The AI neural network learns to remove noise step-by-step:

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t) \right)$$

- **Intervention Conditioning:** We condition the denoising process on candidate actions c (e.g. $c = \text{"Throttle CPU 15\%"}$). The model outputs a synthetic 20-step time-series showing exactly how temperature, OSNR, and load will behave into the future under action c .

Algorithm 3: Shapley Value Root-Cause Attribution

- **Role:** Fairly determines how much each metric contributed to an anomaly.
- **Why this over simple correlation/regression?**
- Standard correlation confuses cause and effect (e.g. high fan speed correlates with high temp, but fan speed isn't the cause).
- Shapley values come from **Cooperative Game Theory** (Nobel Prize winning concept by Lloyd Shapley). They evaluate every possible subset of features to measure a feature's true marginal contribution.

- **Formula:**

$$\phi_i(v) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(|N|-|S|-1)!}{|N|!} \big[v(S \cup \{i\}) - v(S)\big]$$

- **Properties guaranteed:**

- **Efficiency:** Sum of all feature contributions equals the total anomaly score.
- **Symmetry:** Two features contributing equally get equal attribution.
- **Null Player:** A feature that never impacts the outcome gets 0% weight.

Algorithm 4: VCG (Vickrey-Clarke-Groves) Auction for Task Allocation

- **Role:** Assigns tasks (Diagnostics, Bargaining, Execution) to the best AI agent.
- **Why this over simple heuristic assignment?**
 - In multi-agent systems, agents might "overstate" their suitability to get assigned tasks.
 - VCG is a second-price auction mechanism that guarantees **Dominant-Strategy Incentive Compatibility (DSIC)**: an agent's best strategy is ALWAYS to report its true capability.
- **Payment Formula:**

$$\text{Payment}_i = \sum_{j \neq i} v_j(a_{-i}) - \sum_{j \neq i} v_j(a)$$

- **Example in CTG-CPM:**

- Agent 1 bids 98 on Diagnostics $\rightarrow$ Assigned Diagnostics.
- Agent 2 bids 95 on Bargaining $\rightarrow$ Assigned Bargaining.
- Agent 3 bids 99 on Execution $\rightarrow$ Assigned Execution.

Algorithm 5: Extensive-Form Game & Backward Induction (SPE)

- **Role:** Models resource bargaining between the system host (P2) and the remediation agent (P1).
- **Game Setup:**
- **P2 (Hardware/System)** chooses Investment level:
 - *High Investment:* Surplus = 18, Cost to P2 = 2.
 - *Low Investment:* Surplus = 14, Cost to P2 = 1.
- **P1 (Remediation Agent)** proposes Split:
 - *Fair:* Equal 50/50 split of surplus.
 - *Greedy:* P1 takes Surplus - 1, P2 gets 1.
- **P2 (System Response)** responds:
 - *Accept:* Both get shares, P2 pays cost.
 - *Reject:* Surplus destroyed (0), P2 still pays cost.
- **Backward Induction Solution (Step-by-Step):**

- **Terminal Nodes (P2 Response):**
 - High & Fair: Accept gives \$7\$, Reject gives \$-2\$ $\rightarrow$ **Accept**
 - High & Greedy: Accept gives \$-1\$, Reject gives \$-2\$ $\rightarrow$ **Accept**
 - Low & Fair: Accept gives \$6\$, Reject gives \$-1\$ $\rightarrow$ **Accept**
 - Low & Greedy: Accept gives \$0\$, Reject gives \$-1\$ $\rightarrow$ **Accept**
- **P1 Proposal Choice:**
 - Under High Investment: Greedy gives P1 \$17\$, Fair gives \$9\$ $\rightarrow$ **Greedy**
 - Under Low Investment: Greedy gives P1 \$13\$, Fair gives \$7\$ $\rightarrow$ **Greedy**
- **P2 Initial Choice:**
 - High Investment leads to Greedy Accept $\rightarrow$ Net P2 Payoff: \$-1\$
 - Low Investment leads to Greedy Accept $\rightarrow$ Net P2 Payoff: \$0\$
 - P2 compares \$0 > -1\$ $\rightarrow$ **P2 chooses Low Investment.**
- **Subgame Perfect Equilibrium (SPE) Path:**

$$\text{Path} = (\text{Low Investment}, \text{Greedy Proposal}, \text{Accept}) \implies \text{Payoff: } (u_1 = 13.0, u_2 = 0.0)$$

Algorithm 6: Nash Equilibrium Conflict Resolution

- **Role:** Prevents two remediation agents from taking conflicting actions (e.g. Agent A trying to load-balance traffic to Node 2 while Agent B takes Node 2 offline for maintenance).
- **How it works:**
 - Formulates a 2×2 payoff matrix between Agent A and Agent B strategies.
 - Uses **Best-Response Analysis**: finds cell (r, c) where Agent A's action is optimal given B's choice AND B's action is optimal given A's choice.
 - Result: Guaranteed pure-strategy Nash Equilibrium with zero strategy conflicts.

Algorithm 7: LLM-Powered Diagnostics (Groq Engine)

- **Role:** Converts complex telemetry, game theory math, and counterfactual scores into clear, human-understandable JSON diagnostics.
- **LLM Used:** Groq Cloud API with [openai/gpt-oss-20b](#) model.
- **Why Groq?** Groq's LPU (Language Processing Unit) architecture processes over 500 tokens per second, returning complete LLM analysis in **under 0.8 seconds** (maintaining the sub-second MTTR target).
- **Structured JSON Schema Output:**

```
{
  "severity": "High",
  "problem_title": "CPU and Memory Saturation",
  "problem_description": "CPU utilization is at 92.5% with memory at 78.3%. This indicates thermal stress...",
  "root_cause": "High CPU workload (50% Shapley attribution) combined with memory pressure...",
  "risk_if_unresolved": "Thermal throttling and system crashes within 2-4 hours...",
  "remediation_steps": [
    {"step_number": 1, "action": "Identify High CPU Processes", "detail": "...", "expected_impact": "..."},
    {"step_number": 2, "action": "Apply CPU Throttling 85%", "detail": "...", "expected_impact": "..."},
    {"step_number": 3, "action": "Optimize Memory Cache", "detail": "...", "expected_impact": "..."}
  ],
  "expected_outcome": "System health score improves from 24.5 to 83.0/100.",
  "truck_roll_avoided": true,
  "estimated_fix_time": "< 10 seconds"
}
```

5. End-to-End Execution Flow (Step-by-Step Scenario)

Here is what happens inside the system during a real incident:

```
[TELEMETRY STREAM] → [ANOMALY DETECTED] → [GENERATE WHAT-IF] → [GAME THEORY & LLM] → [AUTONOMOUS FIX]
```

Incident Scenario: 5G Optical Fiber Transceiver Degradation

• Step 1: Detection (~1s)

- Telemetry collector reads OSNR = 16.87 dB (below the 18.0 dB threshold).
- GraphSAGE GNN flags node `opt-transceiver-5g-01` as degraded and identifies 3 downstream nodes at cascade risk.

• Step 2: Generation (~2s)

- Time-Series Diffusion model synthesizes 4 parallel future streams:
- *Scenario 1 (Status Quo)*: Health Score = 24.5 (Unstable, OSNR drops to 10 dB)
- *Scenario 2 (Adjust Laser Bias)*: Health Score = 85.0 (Stabilized, OSNR > 20 dB)
- *Scenario 3 (Throttle 15%)*: Health Score = 83.0 (Stabilized)
- *Scenario 4 (Reroute)*: Health Score = 78.0 (Stabilized)

• Step 3: Simulation & Negotiation (~2s)

- VCG Auction assigns task roles to Agent 1 (Diagnostics), Agent 2 (Bargaining), Agent 3 (Execution).
- Agent 1 calculates Shapley Values: Laser degradation = 40%, Temp = 35%, OSNR = 15%, Packet Loss = 10%.

- Agent 2 solves Backward Induction SPE $\rightarrow$ selects ([Low Investment](#), [Greedy](#), [Accept](#)).
- Groq LLM generates structured problem summary and 3-step remediation instructions.
- Agent 3 tests the fix in the Digital Twin sandbox $\rightarrow$ confirms system stabilizes.
- **Step 4: Autonomous Deployment (~5s)**
- System generates NETCONF XML configuration script:

```
<interface name='opt-transceiver-5g-01'>
<laser-bias>current-adjusted</laser-bias>
<traffic-policy>load-balance-30</traffic-policy>
</interface>
```

- Script pushed to device via API.
- Status updated on Web Dashboard.
- **Total Elapsed Time: < 10 seconds. Zero truck roll required.**

6. Codebase Map & Module Reference

```
d:\Predictive Maintenance Project 3\
■■■ game_engine.py # Game Theory (SPE, VCG, Shapley, Nash)
■■■ telemetry_collector.py # Live psutil host & 5G transceiver telemetry
■■■ counterfactual_engine.py # Diffusion "What-If" time-series generator
■■■ agentic_remediator.py # 3-Agent self-healing orchestrator
■■■ llm_diagnostician.py # Groq LLM diagnosis & JSON parser
■■■ app.py # Flask web server REST APIs
■■■ main.py # CLI controller & demo runner
■■■ test_prototype.py # Automated test suite (5 tests, 100% pass)
■■■ templates/
■ ■■■ index.html # Premium glassmorphism Web UI
■■■ .env.example # API key config template
■■■ generate_review2_ppt.py # PowerPoint generator script
```

7. Viva & Review Q&A Cheatsheet (Top 15 Questions & Answers)

Q1: Why did you use Counterfactual Telemetry instead of standard predictive ML?

Answer: Standard predictive ML only tells you *that* a failure will happen. It doesn't tell you *what will happen if you take action A vs action B*. Counterfactual telemetry generates hypothetical future time-series data under different candidate actions ("What-If" scenarios), enabling *prescriptive* self-healing before touching live infrastructure.

Q2: How does your system guarantee "Zero-Risk Remediation"?

Answer: Every candidate fix script is tested against the GenAI counterfactual telemetry stream inside a Digital Twin Sandbox first. Only if the simulation proves that the fix stabilizes system health score above the safety threshold is it approved for live deployment.

Q3: Why is VCG Auction used for multi-agent task allocation?

Answer: VCG (Vickrey-Clarke-Groves) is a mathematically proven auction mechanism that guarantees Dominant-Strategy Incentive Compatibility (DSIC). This means agents are incentivized to bid their true capability scores without cheating, ensuring globally optimal task assignment.

Q4: Explain the Subgame Perfect Equilibrium (SPE) outcome in your game tree.

Answer: Using Backward Induction, Player 2 (System) prefers Accept at all final nodes. Player 1 (Remediation Agent) prefers Greedy proposals over Fair proposals. Backward induction shows that High Investment gives P2 a net payoff of $-\$1$, while Low Investment gives $\$0$. Since $\$0 > -\1 , P2 chooses Low Investment, leading to the SPE equilibrium ([Low Investment](#), [Greedy Proposal](#), [Accept](#)) with final payoff $$(13, 0)$$.

Q5: How do Shapley Values help in Root Cause Analysis (RCA)?

Answer: Shapley values come from cooperative game theory. Instead of simple correlation, Shapley values evaluate the marginal contribution of a metric across all possible combinations of telemetry features. This gives an exact, axiomatically fair percentage weight (e.g. CPU 50%, Temp 30%) to each root cause feature.

Q6: Why did you use GraphSAGE GNN instead of standard CNNs?

Answer: Network topologies are non-Euclidean graphs (nodes connected by dynamic links). Standard CNNs require regular grid structures like images. GraphSAGE aggregates node neighbor embeddings to capture topological connectivity and cascading failure propagation.

Q7: Why use Diffusion Models for time-series generation instead of GANs?

Answer: GANs suffer from mode collapse and training instability on time-series metrics. Time-Series Diffusion (Diffusion-TS) uses denoising score matching, which is stable, avoids mode collapse, and allows precise conditioning on intervention variables.

Q8: Why did you choose Groq API over running a local LLM?

Answer: Local LLMs (like Ollama on consumer hardware) take 5 to 15 seconds to generate text, breaking our < 10 -second MTTR target. Groq's LPU hardware processes 500+ tokens/sec, completing full JSON diagnostic generation in under 0.8 seconds.

Q9: What happens if the Groq LLM API fails or goes offline?

Answer: The system has a built-in [_generate_fallback_diagnosis\(\)](#) engine. If the LLM call fails, the system automatically falls back to rule-based Shapley and game theory analysis, ensuring 100% uptime without crashing.

Q10: How does CTG-CPM reduce OPEX by 40%?

Answer: By discovering software and configuration fixes (e.g., laser bias adjustment or CPU throttling) that extend equipment life by 6+ months, CTG-CPM eliminates unnecessary physical hardware replacements ("truck rolls"), which account for up to 40% of operational expenses in telecom and data centers.

Q11: What is the difference between Live Host mode and 5G Network mode in your prototype?

Answer: Live Host mode ingests real-time CPU, RAM, Disk I/O, and process telemetry from the user's laptop using `psutil`. 5G Network mode simulates an optical backhaul transceiver experiencing OSNR degradation. Both feed into the exact same 5-layer self-healing pipeline.

Q12: How fast does your multi-agent decision pipeline execute?

Answer: In our benchmark tests (`main.py` and `test_prototype.py`), the full multi-agent decision pipeline (VCG + Shapley + SPE + Nash + Digital Twin) executes in **under 0.5 milliseconds (0.0005 seconds)**.

Q13: What script formats are generated for execution?

Answer: For 5G telecom networks, it generates **NETCONF/YANG XML** configuration payloads. For host laptop systems, it generates **PowerShell commands** for process priority demotion and CPU frequency capping.

Q14: How does Nash Equilibrium prevent agent conflicts?

Answer: When two agents propose remediation strategies, we construct a 2-player normal-form payoff matrix. Using best-response analysis, we find the pure-strategy Nash Equilibrium where neither agent can improve by changing action, preventing conflicting actions (e.g., load-balancing to a node being taken offline).

Q15: Is your codebase publicly accessible for verification?

Answer: Yes! The complete codebase (15 files, 3,368+ lines of code, unit tests) is committed to GitHub at <https://github.com/sagnikbasutaan2004-source/CTG-CPM-Self-Healing-Networks>.

8. Individual Contributions Breakdown

Team Member	Reg No	Contribution %	Key Deliverables & Responsibilities
-------------	--------	----------------	-------------------------------------

Sagnik Basu	23MID0042	33.3%	• Overall System Architecture & 5-Layer Stack Design • Game Theory Engine (game_engine.py: VCG, Shapley, SPE, Nash) • Groq LLM Diagnostics Integration (llm_diagnostician.py) & JSON parsing • GitHub Repo setup, Git security audit & automated unit testing (test_prototype.py)
C Sriharsha	23MID0111	33.3%	• Telemetry Collection Engine (telemetry_collector.py) • Real-time psutil host metric ingestion (CPU, RAM, Disk, Processes) • 5G Optical Transceiver anomaly generator • CLI Orchestrator (main.py) & sub-second pipeline timing benchmarks
Maitree Singh	23MID0076	33.3%	• Generative Counterfactual Engine (counterfactual_engine.py) for "What-If" streams • Multi-Agent Remediation Orchestrator (agentic_remediator.py) & Digital Twin Sandbox • HMW-Style Modern Glassmorphism Web App UI (templates/index.html) • Flask Server REST API endpoints (app.py)

Document compiled for Review 2 & Viva Defense — CTG-CPM Self-Healing Networks Project.